

VendorPulse — Agent Hosting Options: Pros & Cons

The three ways to run the agent layer on Microsoft Foundry, compared. All three use the same GPT-4o model on the Foundry Responses API inside Shell's tenant — the difference is **who orchestrates the agent and who owns the prompt**. Our recommendation: **Option 3 (MAF SDK in-process)** now, with **Option 2 (Hosted Agents)** as a fast-follow once GA. Read through the lens of our two non-negotiables: the **human-approval gate (HITL)** and **governance/auditability**.

At-a-glance comparison

Dimension	1 · Portal Prompt / Agent-Service	2 · Foundry Hosted Agents	3 · MAF SDK in-process (chosen)
Who runs the agent code	Foundry	Foundry (managed compute)	Our backend (the VM/App Service)
Who owns the prompt	Foundry assembles it	You (your code)	You (your code)
HITL / approval gate	■■■ Fights the "act" default	■ Preserved (your code)	■ Architectural — agent has no send tool
Governance & audit (IRM 3.492 / EU AI Act)	■■■ Prompt not fully shown to auditors	■ Prompt in Git, verbatim	■ Prompt in Git, verbatim
Deterministic-first core	■■■ Assumes LLM drives	■ Unchanged (your code)	■ Unchanged (your code)
Status	GA	Public preview (GA expected early Jul 2026)	GA
Fit with the 4-week plan	■■■ Rewrite (+2–3 wks)	~+1 wk + GA/region dependency	■ On plan
Infra burden	None (Foundry-managed)	Foundry-managed endpoint	We own the runtime

■ strength · ■■■ caution

Option 1 — Foundry Prompt / Agent-Service agents (built in the portal)

Pros

- Generally available; nothing to host or operate (Foundry-managed).
- Fast to stand up simple agents; built-in tool types (OpenAPI, MCP, file search, code interpreter).
- Microsoft-blessed "managed agent" path that the client may already favour.

Cons (decisive for a governance product)

- Prompt ownership lost:** Foundry assembles the instruction context — you can't show auditors "this exact prompt, every time" (weakens IRM 3.492 / EU AI Act transparency).
- HITL friction:** Agent-Service is built to *act* autonomously; you must re-engineer and re-prove the human-approval gate against that default every release.
- Dilutes deterministic-first:** the model assumes the agent drives the process, undercutting the "AI only drafts text, code makes every decision" story.
- Lock-in & timeline:** logic lives in portal config (outside your CI); migrating away is hard, and adopting it is effectively an agent-layer rewrite (+2–3 weeks).

Option 2 — Foundry Hosted Agents (your MAF code, hosted by Foundry)

Pros

- Same code as Option 3** — no agent rewrite; HITL gate, deterministic core, and prompt ownership all preserved.
- Foundry-managed compute: autoscale, per-agent Entra identity, session state, built-in tracing.
- Supports BYO-VNet / no public egress — good for Shell network controls.

Cons

- Public preview today** (GA expected early July 2026) — a preview dependency is normally a blocker for Shell IRM / TRB production sign-off.
- Region/residency check:** preview regions include Sweden Central (EU) but **not confirmed West Europe** — must verify before committing prod.
- Not a one-click move: agents must be packaged and their tools/data (Graph, Postgres) made reachable from the hosted runtime (~+1 week).

Verdict: the right managed option — but as a fast-follow once GA, not for this build.

Option 3 — MAF SDK in-process (*recommended baseline*)

Pros

- bullet **Full prompt ownership** — every token in Git, versioned, shown to auditors verbatim (strongest IRM 3.492 / EU AI Act posture).
- bullet **HITL is architectural** — the agent literally has no send tool; sending is a separate deterministic route after a human approves. Nothing to "configure around."
- bullet **Deterministic-first preserved** — all decisions remain plain code; `ENABLE_LLM=false` fallback still works.
- bullet **GA, on-timeline, easiest to certify** — all logic in your CI and scanners; hits the 4-week plan; runs where your backend runs (data residency).
- bullet **Keeps options open** — lifts onto Hosted Agents (Option 2) as a ~1-week hosting swap; LLMProvider abstraction keeps Anthropic selectable.

Cons

- bullet **We own the runtime** — we manage the container/scaling (modest for a steady internal workload; Hosted Agents removes this later).
- bullet **Slightly more of our own orchestration code** than a fully-managed agent (but that code is exactly what gives the audit transparency).

Our two key points — keep these front and centre

Human-in-the-loop (HITL): VendorPulse's safety model is the agent drafts, a human approves, then deterministic code acts. In-process, that gate is **built into the architecture** — the agent has no ability to send. Portal agents are designed to act autonomously, so they work against this gate. This is the single biggest reason to avoid Option 1.

Governance & auditability (IRM 3.492 / EU AI Act): auditors must see the **exact prompt and the exact decision path**. With our code (Options 2 & 3), the prompt is in Git and every decision is deterministic and logged. With portal agents (Option 1), Foundry assembles the prompt — you lose the verbatim transparency the regulation expects.

Bottom line: Ship on **Option 3 (MAF in-process)** — GA, on-plan, fully auditable, HITL by design. Adopt **Option 2 (Hosted Agents)** as a fast-follow after its July GA — same code. Steer away from **Option 1**, the only path that costs you both prompt ownership and your approval gate.